

Real-Time Systems – QNX Laboratory Exercise 1 (week 2)

Lecturer: Dr. Pham Viet Hung

Lab notes: These tasks are to be conducted individually (but you should discuss with your friend, don't get lost alone). Although you don't need to submit a report for assessment, you should make a **note document** and keep your solution/s **code (*.c file/s)**, so that you can review later when doing the final project, as well as prepare for the In-class Tests.

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. You should have screen captures of the code running in:

1. Momentics.
2. The VMplayer QNX x86 Target VM.
3. A Telnet terminal connected to the QNX x86 Target VM.

Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. **HINT:** When deploying your code via a Telnet connection, make sure the name of your program is unique (or it is in a unique directory under /tmp/yourname) and that it displays your student number, full name and list of the resulting cards. You should also display the hostname of the target and the process ID.

Finally, keep the source files (*.c files) together with the note.

Introduction

Make sure that you finish the Lab_0. You should consult with the following files from the EEET2588 Canvas shell:

[Getting_Started_with_QNX_at_RMIT.PDF](#)

Once you have loaded and booted into the Ubuntu desktop, you can start the **QNX QNX Momentics Tool Suite** (which is the IDE we will be using throughout the semester) as well as a **QNX x86 Target**. You should familiarise yourself with the Ubuntu platform, VirtualBox/VMware Player, QNX Momentics Tool Suite and the various commands used by the QNX Target using the above documents. In particular make sure that you know how to start the Ubuntu File Manager, start QNX Target VMs, start a QNX Momentics project and use the QNX Momentics inbuilt Help system.

1. Exploring the provided sample project:

Read at least section 1 through to 10 of the [Getting_Started_with_QNX_at_RMIT.PDF](#). Make sure you can launch the provided sample code to both of the local QNX Target VMs (VM_x86_Target01 and VM_x86_Target02) and confirm that you understand how the sample C-code relates to the console output. You can go to Lab 1 on Canvas to find the sample.c code.

Remember to press the stop button and the crosses to clear the console log between each successive launch   . It is good habit to press the double cross button after your program has finished executing as it will terminate the program and close all network connections to the launched thread on the target.

TASK 1A: Add the following example 'for loop' to the end of the sample C project file:

```
int i = 0;
for (i=1; i <= 10; i++)
```

```

{
    printf("Loop iteration %d\n", i);
    sleep(2); //sleep for 2 seconds
}

```

The code starts with integer *i* set to 1, repeats the loop while *i* ≤ 10, and increments *i* (*i*++) each time the loop is executed. Add the above code to the main body of the sample code and run it on both QNX VMs. If you want to find out more about the `printf()` function (or any other) you can look it up in the QNX Help (under Help->Help Contents).

Else you can read the 'Beej's Guide to C Programming' ([bgc_A4.pdf](#) in the course's Canvas shell) or refer to the "[QNX_Neutrino_RTOS_C_Library_Reference.pdf](#)" located in the "QNX Documentation" folder on the Ubuntu desktop. Note: The 'for' loop in the above example could easily be replaced with a while loop, which takes the form:

```

while (expression)
    statement;

```

where the while loop repeats the statement while the expression is non-zero (true).

As the loop executes the `sleep(2)` function 10 times, your code should execute for just over 20 seconds. While the code is executing, click in the terminal window of the QNX Target VM and type "pidin" at the console prompt.

```
# pidin
```

Answer the following questions in your report:

Make sure you identify the process number of the thread in the list displayed by the QNX Target.

What is it? _____ and is it the same as the process ID reported back in Momentics? _____

Chances are that the process will be blocked in the NANOSLEEP state. What line of code is it most likely executing? _____

While you are at the QNX prompt, type the command:

```
# ifconfig
```

The `ifconfig` command will return details about the QNX Target's network adaptor. Confirm that the first QNX Target (VM_x86_Target01) is at IP address ____:____:____:____. Similarly, the second (VM_x86_Target02) target's IP address should be different at ____:____:____:____. You should be able to relate the IP address to the launch macros settings and the console header status bar when you launch the code in Momentics. Keep in mind that the IP address can change depending on network setup and how VMware player is installed.

Write the MAC address for each of the two x86 QNX Target VMs that you are using:

VM_x86_Target01 MAC address: ____:____:____:____:____:____ and IP address: ____:____:____:____

VM_x86_Target02 MAC address: ____:____:____:____:____:____ and IP address: ____:____:____:____

While you are at the QNX prompt, type the command:

```
# hostname
```

Write the hostname for each of the two x86 QNX Target VMs that that you are using:

VM_x86_Target01 hostname is: _____

VM_x86_Target02 hostname is: _____

NOTE: Network setting should be NAT.

NOTE: The **independent QNX x86 Target VMs** have been setup by default to be Bridged, meaning they will request their IP from the DHCP server (e.g. the router) on the physical network. If more than one of the same **independent QNX x86 Target VMs** is run on the same physical network, then a network clash will occur if the VMs have the same MAC address and/or hostname.

Once you have confirmed the IP address of both QNX Target VMs you can extend your code in Momentics to include a simple if-else statement inside the loop:

```
if (i > 5)
    printf ("Too large\n");
else
    printf ("Acceptable\n");
```

Once you are happy with the result, modify the condition in the 'if statement' so that each time the value of i (loop index) is printed, the appropriate phrase of 'odd' or 'even' is also printed.

Hint: % is the modulus (remainder) operator in C, so $i\%2$ is 0 (false) if i is even and $i\%2$ is 1 (true) if i is odd. This could also come in handy for the last task.

The sample project that is provided within the Live Image is setup to only compile code for x86 based architectures. The next section will walk you through the process of compiling code for x86 and ARMv7 architectures. However, by this stage you should know where your project files are saved, how to copy and save a QNX project to and from a USB storage device, how to enter, edit, compile and launch your code to a QNX Target at a given IP address on the Ubuntu VM's internal (local) network.

2. Write your first QNX program

TASK 2A:

Close Momentics and re-start it. When it asks you to select a workspace directory, press 'browse' and go back to the QNX7workspace directory. Create a new folder and press OK twice.

You can create a new project now. Briefly, this involves right clicking in the left hand (Project Explorer) panel and selecting: 'New'-'>'QNX Project'. The select QNX Executable, press Next and type a suitable project name and select the appropriate 'CPU Variants'. In this course we will only be using x86_64 build types and press Finish. You now should see a basic C file with the same name as the project name you selected.

Now you need to add a QNX target to your project, learn how to compile your code for x86 and targets and launch and debug your code using the debug features.

Here is a simple example of defining and calling a function in C (from Beej's Guide):

```
int plus_one(int n) /* Function Definition */
{
    return n + 1;
}
```

```

int main(void)
{
    int i = 10, j;
    j = plus_one(i); /* Function Call */
    printf("i + 1 is %d\n", j);
    return 0;
}

```

Modify the above code to work within QNX. Once you have it working, extend the code so that the function converts a temperature given in degrees Celsius to degrees Fahrenheit. (The conversion formula is $F = C * 9/5 + 32$)

Test and debug your code by creating a new launch configuration to launch the debug version of your code.

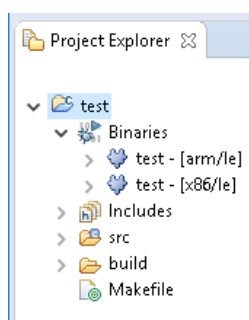


Image showing the x86_64 binary files inside the QNX Momentics project explore.

You can choose to run the binaries in debug mode or run mode by using the drop-down box. It is recommended that you learn how to use the debug mode when you are developing your solutions. Notice that when you select debug mode the perspective is changed in Momentics to the Debug view. You can change back to the C/C++ perspective by pressing the C perspective

button on the top right-hand side of the screen:

If the quick select icons are not available, you can change perspectives by selecting:

- Window->Perspective->Open Perspective->QNX System Information (use to add targets)
- Window->Perspective->Open Perspective->C/C++ (default – programming mode)
- Window->Perspective->Open Perspective->Debug (debug mode)

When in the Debug perspective, use the F5 key to “Step Into” and F6 key to “Step Over” a function call. Pay particular attention to the variables and their values in the top right panel in the debug view.

Once you are satisfied that you understand how to change perspective, add QNX targets and debug your code, create a new launch configuration that will launch the x86_64 version of your code one to one of the provided QNX VMs.

You might want to add some of the code provided in the sample project which lists the hostname and number of available CPUs to your program.

If you use lab computer, always save the project directory to your USB or share drive.

If you are familiar with C (or C++) you should not find the following exercises too challenging, however keep in mind that the main task is to familiarize yourself with compiling code for x86 architectures and executing and debugging programs on networked QNX Targets. Create a separate project for each of the following problems within the same workspace.

3. Working with Structures

A structure (struct in c) is used to group several variables (types) together. Here is a simple example defining and using a structure (from Beej's Guide):

```
#include <stdlib.h>
#include <stdio.h>

/* Here we declare the type so we can use it later: */
struct stuff {
    int val;
    float b;
};
/* Note that we don't actually have any variables of that type, yet. */

int main(void)
{
    /* ok, now let's declare a variable "s" of type "struct stuff" */
    struct stuff s;
    s.val = 3490; // assignment into a struct
    s.b = 3.14159;
    printf("The val member in struct s is: %d\n", s.val);
    return 0;
}
```

Note that structures are often accessed using pointers (see Beej's Guide).

TASK 3A: Use the program code above as a starting point and write a function that takes the struct as an argument. Make sure you can execute the code on QNX Targets using the pre-defined marcos. You first might need to copy the executable file to the /tmp/yourname directory on the QNX Target using the "Target File System Navigator" or similar. You can copy the executable file to a QNX Target by dragging it from the Project Explorer panel (left hand side) and dropping it in the directory in Target File System Navigator panel (bottom of the screen) in Momentics.

Remember to execute the program at the prompt by typing: ./filename

4. Working with Arrays and User Input

Here is an example of reading from the keyboard (from Beej's Guide):

```
#include <stdlib.h>
#include <stdio.h>

int main(void)
{
    int i, num, result = 0;
    scanf("%d", &num); // read the number from the keyboard
    for(i = 1; i <= num; i++) // compute the sum of numbers from 1 to num
    {
        result += i;
    }

    printf("%d\n", result); // output the result
}
```

```
        return 0;
    }
```

Task 4A: Write a small program with the above code and confirm you can enter user input via the Momentics console.

Once you have done that, now try executing the code directly on the target and enter the text in via the QNX VM terminal. You can upload the binary to the target via the Target System File Navigator. To do this select:

Window->Show View->Target System File Navigator

Select the IP address of the QNX target in the bottom panel and navigate to the /tmp directory. Now you can drag and drop the appropriate 'Binary' file from the project tree (left hand panel) to the /tmp directory. Then go to the QNX VM terminal, navigate to the /tmp directory (# cd /tmp) and execute the file by typing: # ./myBinary where the myBinary is the name of the binary file you drag and dropped into the /tmp directory. If available, try also doing this on the ARMv7le targets too, however you will need to use the arm/le binary file and will need to telnet to the target to get a # prompt.

Task 4B: Write a program that defines and initialises an array holding 10 numbers, then reads a number from the keyboard, and prints that number of the entries in the array. You can use the below code which shows an example of defining and using an array in C:

```
int i;
int age[5] = {10,20,25,8,2}; // initialization of whole array in definition
for(i = 0; i < 5; i++)
{
    printf("age %d is %d\n", i, age[i]);
}
```

Note that there are 5 elements in the array, but the array index goes from 0 to 4.

5. Working with Chars and Strings

In C a single character can be defined and initialised as:

```
char ch = 'A'; // Note single quotes for a character literal
```

you can use %c in printf to print it:

```
printf("char is: %c\n", ch);
```

An array of char's can be used to hold a string of characters, which can be declared and initialised as:

```
char str[21] = "This is a string!";
```

The extra character is to hold the null character that terminates the string. It can be printed using printf as:

```
printf("String is: %s\n", str);
```

Note: #include <iostream> will not work in C projects. You will need to make a C++ project to use cin and cout objects, however C++ is not supported on all QNX Targets by default.

6. TASK 6A:

Now that you have done all the above tasks, its time to put it into practice by developing a small C program. Your task is to define a structure to describe a playing card.

Example code:

```
typedef struct card {  
    char suit[10];  
    int value;  
};
```

Write a program that uses an array to store a complete deck of cards (containing the standard 52 cards, minus the joker) – each card is described by the struct in the example code above. The program should read a number n , from the keyboard, and print every n^{th} card in the deck.

Assume the deck of cards must be constructed with each suite in order from lowest value to highest and the order of the suites should be: Hearts, Diamonds, Clubs and Spades. Therefore, this means you should start constructing the deck of cards starting from Ace of Hearts, Two of Hearts, Three of Hearts... Ten of Heats, Jack of Hearts, Queen of Hearts and King of Hearts. Then do the same for all the Diamonds, Clubs and Spades.

If the user chooses to printout every 13th card the output should be something like:

suit: Hearts	value: King
suit: Diamonds	value: King
suit: Clubs	value: King
suit: Spades	value: King

If the user chooses to printout every 11th card the output should be something like:

suit: Hearts	value: Jack
suit: Diamonds	value: Nine
suit: Clubs	value: Seven
suit: Spades	value: Five